

Benchmark Report: pyflate

A small link budget, a compressed payload, and a lot of work between the bits

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

Somewhere between a spacecraft and a ground station there is a link budget measured in kilobits.

Compression makes the most of it; decompression turns the arriving bits back into useful data. This benchmark gives that quiet half of the job to Python: read a bit stream, recover symbols, and undo several transformations. Its Huffman matcher looks like the obvious place to start. Measuring it led us to improvements across the whole decoding pipeline.

1. Workload and result

Despite its name, the measured pyflate workload is **bzip2**, not DEFLATE. Each iteration decompresses `interpreter.tar.bz2` from **67,562 bytes to 399,360 bytes** and checks the original MD5. The implementation uses standard-library Python rather than calling a native decompression library. Its main structures are an integer bit buffer, Huffman tables, a move-to-front alphabet, BWT index vectors and output buffers.

One block, six Huffman tables, 148,271 symbols. The decoder switches tables every 50 symbols. Huffman decoding, move-to-front and RUNA/RUNB expansion produce a 336,184-byte L-vector. Inverse BWT and final RLE4 expansion produce the 399,360-byte output.

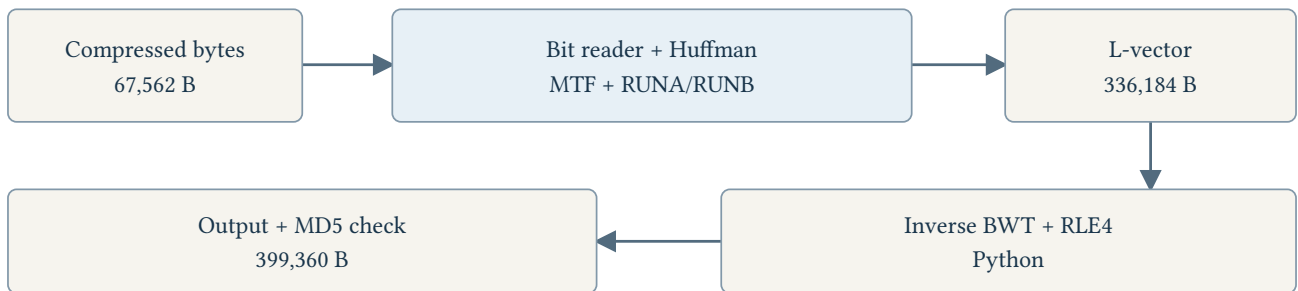


Figure 1: The bzip2 path. RUNA/RUNB and move-to-front are handled together in the symbol loop. The native kernel covers the same combined boundary proposed for hardware.

Configuration	Mean ± SD	vs stock
Stock Python	1,132.50 ± 15.71 ms	1.00×
Optimized Python	288.26 ± 2.71 ms	3.93×

The result is a **74.5% runtime reduction**, exceeding the course's 7% requirement. Output matches `bz2.decompress` byte for byte and passes the unchanged MD5 check. The optimized relative standard deviation is **0.94%**.

Measurement scope. Course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12, pyperformance 1.14.0; 120 measured values per configuration. Source: `results/baseline_pyflate.json` and `optimized_pyflate.json`. Supporting development profiles and ablations use a different environment, labeled on the next page.

2. Profiling and software changes

The stock Python-frame profile highlights Huffman decoding and bit-buffer work, with move-to-front and inverse BWT alongside them. Development instrumentation found a mean of about five entries visited by the symbol matcher, despite its table capacity of 258. A lookup table can remove this scan, but the measured scan is short; per-symbol interpreter work and the later transformations also deserve attention.

bzip2_main: 384 selected samples | inclusive widths

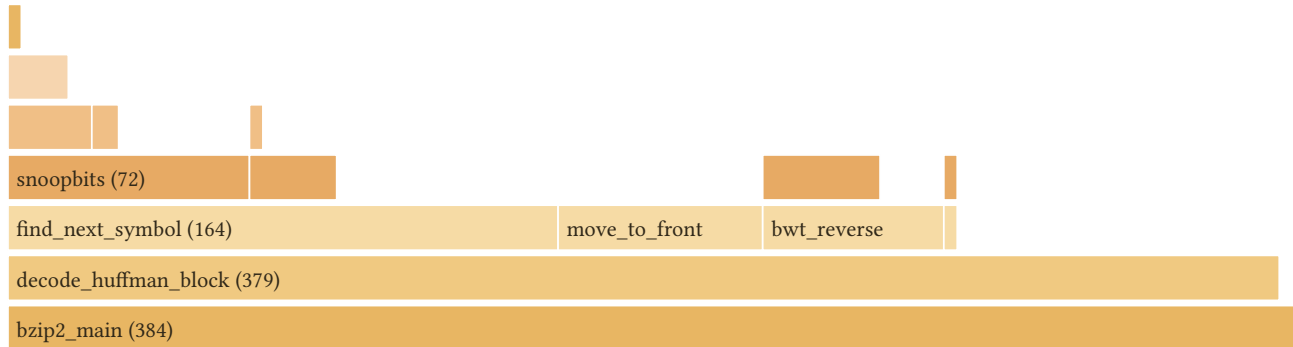


Figure 2: Stock Python call stacks, grouped by function. Width is inclusive samples within `bzip2_main`; nested widths overlap and must not be added. See Appendix A3.

A separate **Windows / CPython 3.12.6** cProfile run gives `find_next_symbol` 0.519 s cumulative out of 1.186 s (43.8%), including its bit-reader callees. `decode_huffman_block` has 0.241 s **self** time and 1.172 s **cumulative** time. These quantities describe different scopes; the parent is not a separate 22% cumulative stage.

Changes that shipped

- **Reduce per-byte overhead:** read the compressed input once, refill the bit window in chunks, bind hot values to locals and accumulate bytes without allocating an object per byte.
- **Decode canonical Huffman codes:** use length-indexed tables and a primary lookup table with a fallback for longer codes. Table construction is amortized over the block.
- **Improve the back end:** replace comparison sorting with counting-sort BWT construction; reverse the MTF list so updates move $O(\text{rank})$ entries; use regex-assisted RLE4 expansion to reduce Python loop iterations. The byte scan/output work is still linear overall.

Component reverted from T3	Runtime	Added cost
None (full T3)	175.5 ms	—
Primary Huffman lookup	185.6 ms	+10.1 ms
Regex-assisted RLE4	207.7 ms	+32.3 ms
Counting-sort BWT	214.0 ms	+38.6 ms

Ablation scope: Windows / CPython 3.12.6, best of seven; dev/pyflate/FINDINGS.md §2b. These are development measurements, not VM timings. Each row reverts one component; the costs need not add. In this experiment, each back-end change contributes more than the primary Huffman table. This motivates optimizing the pipeline rather than the matcher alone.

3. Native execution and the remaining work

The Rust/PyO3 BlockDecoder handles the bit reader, Huffman decode, MTF and RUNA/RUNB expansion. Header parsing, inverse BWT, RLE4 and MD5 remain in Python. One block-level call returns the L-vector and the ending bit position; checking both catches output errors and stream desynchronization.

Same benchmark file	Mean $\pm$ SD	vs Python
Python backend	288.55 $\pm$ 5.13 ms	1.00×
Native decode backend	178.12 $\pm$ 5.46 ms	1.62×

These are 120-value VM measurements from `fallback_pyflate.json` and `native_pyflate.json`. They measure the complete benchmark, not the isolated symbol loop. `HWSW_BACKEND=python|native|auto` selects the implementation; native mode requires the wheel, while auto mode falls back. Pyperf workers need `--inherit-environ HWSW_BACKEND`.

bzip2_main: 97 selected samples | inclusive widths



Figure 3: Optimized **Python** profile, before native offload. Separate matcher frames disappear after inlining; inverse BWT is a substantial remaining component.

Amdahl's law: use one scope and one baseline

For baseline offload fraction f and kernel speedup s , $S = \frac{1}{(1-f) + \frac{f}{s}}$; an infinitely fast kernel gives $S_{\max} = \frac{1}{1-f}$. The fraction must come from the same baseline workload and environment as the speedup. The earlier 25× kernel result and 1.73× cap came from WSL2, as recorded in `rust/pyflate/README.md`. They cannot establish a percentage of the cap for this VM table. Here the supported end-to-end result is **1.62×**; the remaining Python stages limit further improvement at this boundary.

What the inverse-BWT measurements establish

Inverse BWT contains both index-table construction and a dependent traversal. The saved VM phase experiment measures about **137.7 ms** for the whole inverse BWT and **47.8 ms** for the isolated traversal loop. Thus the whole BWT cost cannot be described as a serial pointer chase.

Process-wide counters give the chase process IPC 2.41 and a generic cache-miss/reference ratio of 1.831%. They include setup and other interpreter work, so they do not identify an exact DRAM-access interval or partition cycles into memory and interpreter stalls. The L-vector is 336,184 bytes; the Python index list and its integer objects occupy additional memory. Neither is a 399 KB hardware index table.

A useful next experiment is to port and profile the whole BWT stage in native code. The traversal dependency may then matter more, but its cache level and bottleneck require measurement. The current evidence does not justify a DRAM-side processing-in-memory claim.

4. Hardware acceleration: current proposal

The hardware boundary comprises `huffman_engine` followed by `mtf_cam`. Together they produce the same L-vector as the native decoder. Software keeps block headers, inverse BWT, RLE4 and MD5. A regular decoder datapath and a coarse block-level submission are the main co-design opportunities.

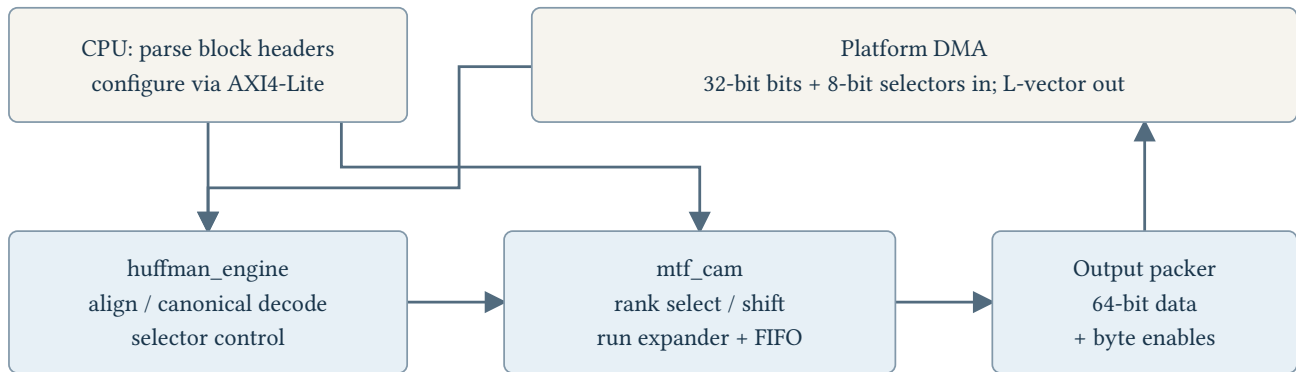


Figure 4: AXI4-Lite configures both modules; AXI4-Stream carries data. The symbol stream stays on chip. Platform DMA returns the L-vector to software.

Huffman engine

A bit aligner feeds a canonical-code comparator/lookup datapath. The selector controller changes the active table every 50 symbols. The current interface specifies a 32-bit compressed input stream, an 8-bit selector stream, and 32-bit output beats containing a 9-bit bzip2 symbol plus type fields. A 32-bit AXI4-Lite window holds controls, `START_BIT`, lengths and counters. It supports up to six tables and a maximum bzip2 code length of 20 bits.

The target is at most 1.10 cycles per symbol, including table construction and switches, with an unstalled benchmark-block model of 153,121 cycles. At the **50 MHz clock target**, that model is 3.06 ms for this module. This is not an achieved frequency or a measured end-to-end hardware runtime.

MTF and run expansion

The input is a **rank**, so the decoder selects an indexed alphabet entry and shifts preceding entries; it does not require a content search across 256 byte comparators. The current proposal maintains up to 256 entries and expands RUNA/RUNB groups through a buffered output packer. The default output width is eight bytes (64-bit data plus byte enables). It targets one MTF symbol per cycle for any valid rank.

Measured mean rank 7.17 characterizes this input, but does not by itself establish a $30\times$ area reduction. The specified area/throughput sweep instead varies output width $W = 4, 8, 16$ and FIFO depth. The default module model is 157,560 cycles, or 3.15 ms at 50 MHz. The two modules stream concurrently; their individual modeled latencies cannot simply be added or substituted for an integrated back-pressure simulation.

Implementation status. Requirements and architecture are approved for both modules.

Microarchitecture, RTL, verification, PPA and integration are still open in the recorded hardware status.

Formal MTF invariants are planned, not completed proofs. Drivers and platform DMA remain integration work. No measured area, clock or power benefit is claimed.

5. Conclusion

The Python changes deliver $3.93\times$ over stock with byte-exact output. Native symbol decoding adds $1.62\times$ over the optimized Python backend, leaving substantial work in BWT and RLE4. The hardware proposal follows that measured boundary; its integrated performance and cost remain to be demonstrated.

Evidence: `hw/huffman_engine/docs/{prd,m}.md`, `hw/mtf_cam/docs/{prd,m}.md`, and `hw/PROGRESS.md` (2026-09-05). Methods and supporting profiles: **report_appendix.pdf**. Repository: [HWSW_Project](#).